

Variable

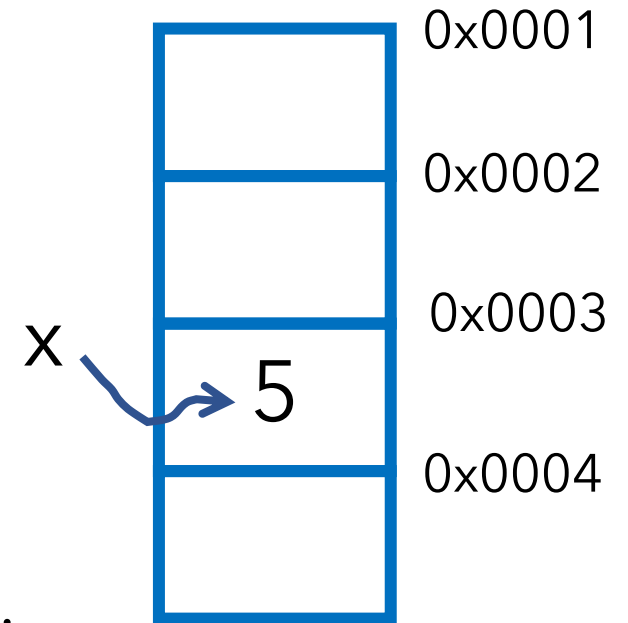
林致宇

Outline

- Introduction to Variables
- Data Types
- Constant
- User Defined Data Types in C++
- Global Variables

Variable

- Variables are a way of **storing** things in a computer program.
- A storage location paired with an associated symbolic name
 - For example, `int x = 5;`
- When do you need variables?
 - If the user has played three times, then ...
 - If the number of coins is larger than 100, then ...



APCS (Taiwan)

- 2017 / 3 / 4

23. 程式執行時，程式中的變數值是存放在

- (A) 記憶體
- (B) 硬碟
- (C) 輸出入裝置
- (D) 匯流排

An Example of Variables

```
1  #include <iostream>
2  using namespace std;
3
4  int main() {
5
6      int number;
7      number = 123;
8
9      int i = 789;
10
11     cout << number << ", " << i << "\n";
12     return 0;
13 }
```

Declaring a variable
(Data Type + Variable Name)

Assignment Operator

Variable Name

- lower camel case (start with a lowercase letter)

- int timesPlayed;
- int numberOfCoins;

- avoid keywords

- public
- Class

- avoid using _

```
1  #include <iostream>
2  using namespace std;
3
4  int main() {
5
6      int 123;
7      int public;
8
9      return 0;
10 }
```

Expected unqualified-id

Expected unqualified-id

Data Types

- Integral Data Type
- Character Data Type
- Floating Point Data Type
- **Boolean Data Type (C++)**
- Type Conversion
- auto (Modern C++)

Data Types

- Integral Data Type
- Character Data Type
- Floating Point Data Type
- **Boolean Data Type (C++)**
- Type Conversion
- auto (Modern C++)

Integral Data Type

Data Type	Minimum	Maximum	bytes
short	-32768	32767	2
int	-2147483648	2147483647	4
long	-9223372036854775808	9223372036854775807	8
unsigned	0	4294967295	4

Integral Data Type

```
#include <iostream>
using namespace std;
```

```
int main(int argc, const char * argv[]) {
```

```
    unsigned int i = 0;
    short j = 0; // 省略 int
    long k = 0; // 省略 int
    unsigned long int l = 0;
```

```
k = 9223372036854775808; // 64-bits
```

2 ⚠ Integer literal is too large to be represe...

```
    cout << k << '\n';
    cout << i << j << l << '\n';
```

```
}
```

-9223372036854775808

000

Program ended with exit code: 0

std::numeric_limits

```
1  #include<iostream>
2  #include<limits>
3  using namespace std;
4
5  int main(int argc, const char * args[]) {
6
7      cout << "The range for type unsigned long is from "
8          << numeric_limits<unsigned long>::min() << " to "
9          << numeric_limits<unsigned long>::max() << endl;
10     return 0;
11 }
```

Class (Class Template)
Chapter 14

Class Method
Chapter 10

sizeof()

```
#include <iostream>
using namespace std;

int main(int argc, const char * argv[]) {

    short j = 0; // 省略 int

    cout << "long: " << sizeof(long) << '\n';
    cout << "short: " << sizeof(j) << '\n';
}
```

long: 8
short: 2
Program ended with exit code: 0

Modern C++

```
4  int main(int argc, const char * args[]) {  
5  
6      int i {8};  
7      int j {3};  
8  
9      cout << i << " , " << j << "\n";  
10     return 0;  
11 }
```

`int i(5);` // 另一種寫法

使用 `{}` 的好處是，當遇到轉型時編譯器會發出警告

```
lincyu@Chih-YudeMacBook-Pro Chapter2 % clang++ -std=c++11 2_16_ModernCPP.cpp  
lincyu@Chih-YudeMacBook-Pro Chapter2 % ./a.out  
8 , 3  
lincyu@Chih-YudeMacBook-Pro Chapter2 %
```

Data Types

- Integral Data Type
- Character Data Type
- Floating Point Data Type
- **Boolean Data Type (C++)**
- Type Conversion
- auto (Modern C++)

Character Data Type

Data Type	bytes
char	1
wchar_t	compiler-dependent and therefore not very portable

Character Data Type

```
1  #include <iostream>
2  using namespace std;
3
4  int main() {
5
6      char c1 = 'B';
7      char c2 = 65;
8      wchar_t c3 = 'C';
9
10     cout << c1 << " , " << c2 << " , " << c3 << "\n";
11     cout << (int)c1 << "\n";
12     cout << sizeof(char) << " , " << sizeof(wchar_t) << "\n";
13
14     return 0;
15 }
```

**B , A , 67
66
1 , 4
Program ended with exit code: 0**

The ASCII Character Set

- https://www.w3schools.com/charsets/ref_html_ascii.asp

Escape Character (跳脫字元)

Escape Sequence	Character
\t	Tab
\n	New Line
\r	Carriage Return
\'	single quote
\"	double quote
\\	backslash

Escape Character (跳脫字元)

```
1  #include<iostream>
2  using namespace std;
3
4  int main(int argc, const char * args[]) {
5
6      cout << "\"Hello\n\tWorld\"\n";
7      return 0;
8  }
```

```
lincyu@Chih-YudeMacBook-Pro Programs % ./a.out
"Hello
    World"
lincyu@Chih-YudeMacBook-Pro Programs %
```

Data Types

- Integral Data Type
- Character Data Type
- Floating Point Data Type
- **Boolean Data Type (C++)**
- Type Conversion
- auto (Modern C++)

Floating Point Data Type

Data Type	bytes
float	4
double	8
long double	16

```
1  #include <iostream>
2  using namespace std;
3
4  int main() {
5
6      float f = 2.5f;
7      double pi = 3.141592653589793;
8
9
10     cout << f << " , " << pi << "\n";
11     cout << sizeof(float) << " , "
12          << sizeof(double) << " , "
13          << sizeof(long double) << "\n";
14     return 0;
15 }
```

2.5 , 3.14159

4 , 8 , 16

Program ended with exit code: 0

lincyu@Chih-YudeMacBook-Pro Programs % ./a.out

2.5 , 3.14159

4 , 8 , 8

lincyu@Chih-YudeMacBook-Pro Programs %

```
1  #include<iostream>
2  #include<iomanip>
3  using namespace std;
4
5  int main(int argc, const char * args[]) {
6
7      float f = 2.568f;
8      double pi = 3.141592653589793;
9
10
11     cout << setprecision(3) << f << " , " << pi << "\n";
12
13     return 0;
14 }
```

[lincyu@Chih-YudeMacBook-Pro Chapter2 % ./a.out
2.57 , 3.14

Data Types

- Integral Data Type
- Character Data Type
- Floating Point Data Type
- **Boolean Data Type (C++)**
- Type Conversion


```
1  #include<iostream>
2  using namespace std;
3
4  int main(int argc, const char * args[]) {
5
6      bool b1, b2, b3;
7      b1 = true;
8      b2 = 0;
9      b3 = 3;
10
11     cout << b1 << " , " << b2 << " , " << b3 << "\n";
12     cout << boolalpha;
13     cout << b1 << " , " << b2 << " , " << b3 << "\n";
14
15     cout << "Sizeof: " << sizeof(bool) << "\n";
16     return 0;
17 }
```

```
lincyu@Chih-YudeMacBook-Pro Programs % ./a.out
1 , 0 , 1
true , false , true
Sizeof: 1
```

→ 讓輸出流把bool type 解析成 true/false

Data Types

- Integral Data Type
- Character Data Type
- Floating Point Data Type
- **Boolean Data Type (C++)**
- Type Conversion
- auto (Modern C++)

Type Conversion

- an integer + a double
 - Promotion
 - $\text{int} + \text{double} \Rightarrow \text{double}$
 - $\text{int} + \text{float} \Rightarrow \text{float}$
 - $\text{int} + \text{long} \Rightarrow \text{long}$
- Type Casting
 - $(\text{double}), (\text{int})$

C-style cast

```
1  #include <iostream>
2  using namespace std;
3
4  int main() {
5
6      int i1 = 3;
7      double d1 = 2.2;
8
9      cout << i1 + d1 << "\n";
10     cout << i1 + (int)d1 << "\n";
11     cout << 5/3 << "\n";
12     cout << 5/(double)3 << "\n";
13     cout << (double)(5/3) << "\n";
14     return 0;
15 }
```

5.2

5

1

1.66667

1

Program ended with exit code: 0

static_cast operator

- `static_cast<type_to_convert_to>(expression)`

```
4  int main(int argc, const char * args[]) {  
5  
6      double d1 {8.6};  
7      double d2 {6.8};  
8      int i = {static_cast<int>(d1) + static_cast<int>(d2)};  
9  
10     cout << d1 << " , " << d2 << " , " << i << std::endl;  
11     return 0;  
12 }
```

延伸閱讀

- https://b8807053.pixnet.net/blog/post/339782825-%5B%E8%BD%89%5D-%5Bc%2B%2B%5D-%E6%A8%99%E6%BA%96%E9%A1%9E%E5%9E%8B%E8%BD%89%E6%8F%9B%EF%BC%9Astatic_cast%2C-dynamic_cast%2C-

Data Types

- Integral Data Type
- Character Data Type
- Floating Point Data Type
- **Boolean Data Type (C++)**
- Type Conversion
- auto (Modern C++)

auto

老師的建議：

這不是 auto 的真正用途，在定義基本型態的變數時，建議要明確地指定型態

```
4  int main(int argc, const char * args[]) {  
5  
6      auto i {8}; // int  
7      auto ul = 168UL; // unsigned long  
8      auto pi {3.1415926}; // double  
9  
10     cout << i << " , " << ul << " , " << pi << std::endl;  
11     return 0;  
12 }
```

說明：

auto 是根據變數的資料來判斷實際的型別，所以若是僅使用 auto 宣告變數，但未定義變數的內容，就會產生錯誤

延伸閱讀

- <https://blog.gtwang.org/programming/cpp-auto-variable-tutorial/>

Constant

Constant

- Literal
- The const Keyword
- Macro

Constant

- Literal
- The const Keyword
- Macro

Literal

```
1  #include <iostream>
2  using namespace std;
3
4  int main() {
5
6      int i = 3;
7      double d = 2.2;
8      char c = '\\';
9
10     cout << i << " , " << d << " , " << c << '\\n';
11     return 0;
12 }
```

Constant

- Literal
- The const Keyword
- Macro

The const Keyword

```
1  #include <iostream>
2  using namespace std;
3
4  int main() {
5
6      int i = 3;
7      const int j = 5;
8
9      i = 6;
10     i = 7;
11     return 0;
12 }
```

❗ Cannot assign to variable 'j' with const-qualified type 'const int'

Constant

- Literal
- The const Keyword
- Macro

Macro

```
#include <iostream>
using namespace std;
#define PI 3.14159
```

12.5664

Program ended with exit code: 0

```
int main(int argc, const char * argv[]) {

    double area = 2 * 2 * PI;
    cout << area << '\n';
}
```

前置處理器並不懂 C++ 語法, 它只是單純執行「文字代換」的工作, 不會做型別檢查

Macro

```
1  #include <iostream>
2  using namespace std;
3
4  #define MAIN main()
5
6  int MAIN {
7      return 0;
8  }
```

User Defined Data Types in C++

User Defined Data Types in C++

- Class
- Structure
 - <https://www.geeksforgeeks.org/structure-vs-class-in-cpp/>
- Union
- **Enumeration**
- **Typedef**

User Defined Data Types in C++

- Class
- Structure
- Union
- **Enumeration**
- **Typedef**

Enumeration

enum 新型別名稱 { 列舉出所有成員};

```
#include <iostream>
using namespace std;
```

```
0, 2
```

```
Program ended with exit code: 0
```

```
int main(int argc, const char * argv[]) {

    enum fruit { apple, banana, orange };
    fruit f1 = apple;
    fruit f2 = orange;

    cout << f1 << ", " << f2 << '\n';

}
```

Enumeration

```
1  #include <iostream>
2  using namespace std;
3
4  int main() {
5
6      enum {red = 5, blue, yellow} color = blue;
7      cout << color << '\n';
8      return 0;
9  }
```

6

Program ended with exit code: 0

Enumeration

```
1  #include<iostream>
2  using namespace std;
3
4  int main(int argc, const char * argv[]) {
5
6      enum class Day {Monday,
7          Tuesday,
8          Wednesday,
9          Thursday,
10         Friday,
11         Saturday,
12         Sunday
13     };
14
15     Day yesterday {Day::Tuesday};
16     Day today {Day::Wednesday};
17
18     cout << static_cast<int>(yesterday) << " , " << static_cast<int>(today) << endl;
19 }
```


User Defined Data Types in C++

- Class
- Structure
- Union
- **Enumeration**
- **Typedef / using**

typedef

```
#include <iostream>
using namespace std;
typedef unsigned char byte;
```

```
int main(int argc, const char * argv[]) {

    byte x = 8;

    cout << x << '\n';
}
```

typedef 型別 新型別名稱;

typedef

```
1  #include<iostream>
2  using namespace std;
3  typedef unsigned char byte;
4
5  typedef struct point {
6      int x;
7      int y;
8  } Point;
9
10 int main(int argc, const char * argv[]) {
11
12     Point p;
13     p.x = 5;
14     p.y = 3;
15     cout << p.x << '\n';
16     return 0;
17 }
```

using (Modern C++)

```
1  #include<iostream>
2  using namespace std;
3  using byte = unsigned char;
4
5  int main(int argc, const char * argv[]) {
6
7      byte x = 63;
8      cout << x << '\n';
9      return 0;
10 }
```

Global Variables

```
1  #include<iostream>
2  using namespace std;
3
4  int x = 5;
5  int y = 3;
6
7  int main(int argc, const char * args[]) {
8
9      int x = 8;
10     int y = 6;
11
12     cout << ::x << " , " << ::y << endl;
13     cout << x << " , " << y << endl;
14 }
```

Practice

- (Practice 1) Swap numbers using temporary variable
- (Practice 2) Change lowercase to uppercase